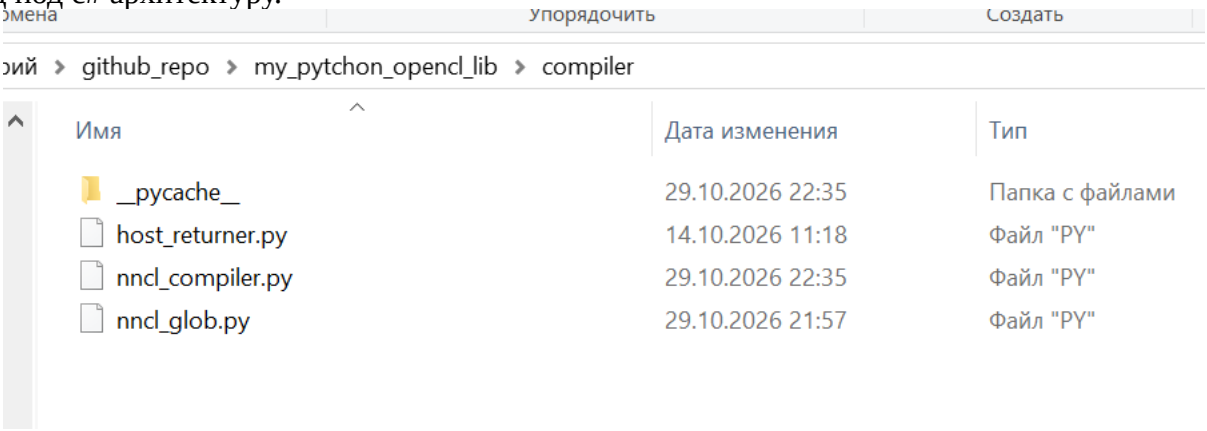
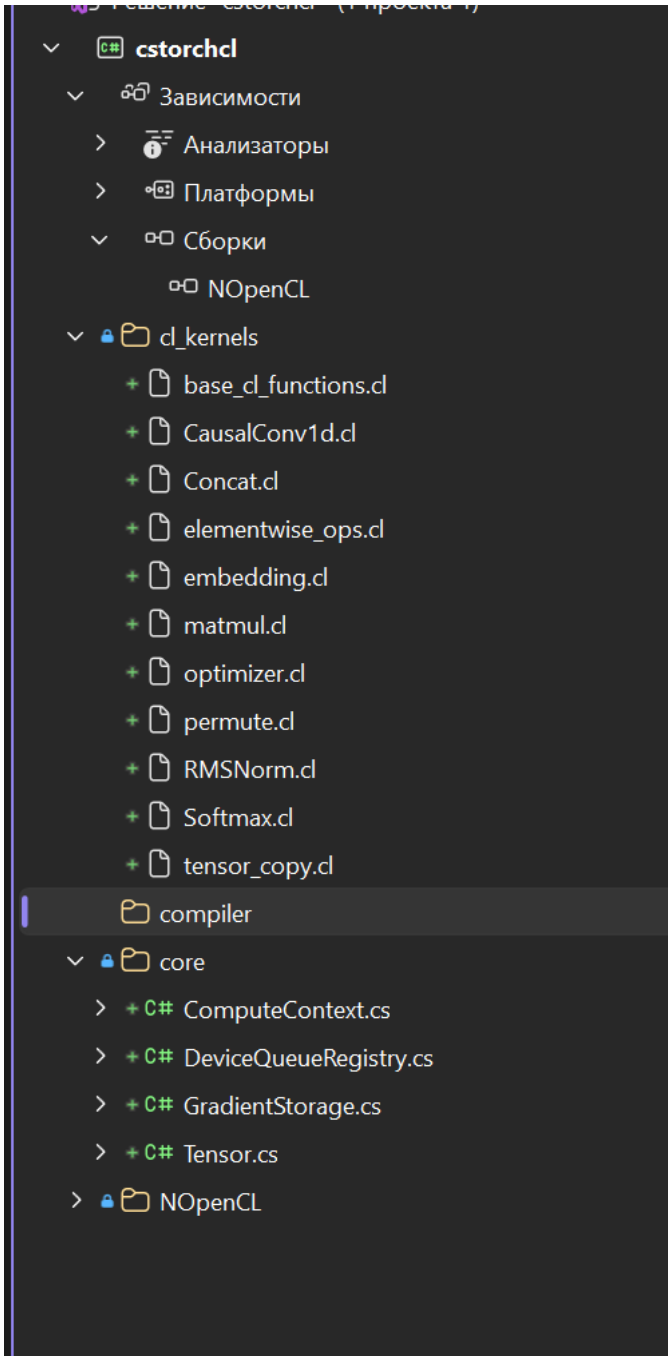
Сначала реализуем логику работы с глобальными объектами. Как я уже говорил у меня есть opencl код ядер, который мы скопирем в c# проект. Нам останется только адаптировать python код под c# архитектуру.



!код отвечающий за компиляцию opencl будет лежать в папке compiler!



Softmax

Softmax.cl

C: > Users > Дмитрий > github_repo > my_pytchon_opencl_lib > cl_kernels > Softmax.cl

```
__kernel void softmax_forward(
    __global const float* X, // Входная матрица (например, скоры внимания) [B * H * L, SEQ_LEN]
    __global float* Y,       // Выходной тензор вероятностей [B * H * L, SEQ_LEN]
    const int seq_len)
{...
}
__kernel void softmax_backward(
    __global const float* grad_output, // Градиент dL/dY [B * H * L, SEQ_LEN]
    __global const float* Y,           // Исходный ВЫХОД Softmax (вероятности) [B * H * L, SEQ_LEN]
    __global float* grad_input,        // Градиент dL/dX [B * H * L, SEQ_LEN]
    const int seq_len)
{...
}
```

Softmax.cl | nncl_glob.py

C: > Users > Дмитрий > github_repo > my_pytchon_opencl_lib > compiler > nncl_glob.py

```python
import os
import numpy as np
import pyopencl as cl
from core.tensor import Tensor
from compiler.nncl_compiler import JITProgramCache
from core.autograd import auto_backward
class NnClGlob:
    def __init__(self, ctx, queue, autograd_ctx=None):
        """
        Диспетчер тяжелых глобальных операций библиотеки nncl.
        Загружает и кэширует скомпилированные Си-ядра.
        """
        self.ctx = ctx

        from core.queue_registry import DeviceQueueRegistry
        self.queue          = DeviceQueueRegistry.get_current()
        self._load_kernels()
        # 3. СОХРАНЯЕМ КОНТЕКСТ ДЛЯ ДЕКОРАТОРА
        self.autograd_ctx = autograd_ctx
    def _load_kernels(self):
        """Загружает Си-код всех тяжелых ядер из cl_kernels"""
        kernels_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'cl_kernels'))

        # 1. Загрузка RMSNorm
        with open(os.path.join(kernels_dir, 'rmsnorm.cl'), 'r', encoding='utf-8') as f:
            self.prg_rmsnorm = JITProgramCache.get_or_compile(self.ctx, f.read())

        # 2. Загрузка Softmax
        with open(os.path.join(kernels_dir, 'Softmax.cl'), 'r', encoding='utf-8') as f:
            self.prg_softmax = JITProgramCache.get_or_compile(self.ctx, f.read())

        # 3. Загрузка Matmul / BMM
        with open(os.path.join(kernels_dir, 'matmul.cl'), 'r', encoding='utf-8') as f:
            self.prg_matmul = JITProgramCache.get_or_compile(self.ctx, f.read())

        # 4. Загрузка Permute (Transpose)
        with open(os.path.join(kernels_dir, 'permute.cl'), 'r', encoding='utf-8') as f:
            self.prg_permute = JITProgramCache.get_or_compile(self.ctx, f.read())
            self.kernel_permute = cl.Kernel(self.prg_permute, "tensor_permute") # Кэшируем само ядро!

        # 5. Загрузка CausalConv1d
        with open(os.path.join(kernels_dir, 'CausalConv1d.cl'), 'r', encoding='utf-8') as f:
            self.prg_conv = JITProgramCache.get_or_compile(self.ctx, f.read())
        # Дописать в метод _load_kernels() класса NnClGlob:
        with open(os.path.join(kernels_dir, 'embedding.cl'), 'r', encoding='utf-8') as f:
            self.prg_embedding = JITProgramCache.get_or_compile(self.ctx, f.read())
    @auto_backward
```

```python
@auto_backward
def softmax(self, x: Tensor, dim: int = -1) -> Tensor:
    """
    Глобальная операция Safe Softmax (Forward)
    Рассчитана на работу со скорами внимания [B * H * L, SEQ_LEN]
    """
    assert dim == -1 or dim == len(x.shape) - 1, "Текущая реализация ядра Softmax поддерживает редукцию только по последней оси."

    seq_len = x.shape[-1]
    len_=seq_len
    print(seq_len)
    num_rows = x.size // seq_len

    # Создаем выходной тензор (память во VRAM выделится автоматически)
    y = Tensor(x.shape, self.ctx, self.queue)

    # Настройка сетки потоков
    local_size = (256,)
    global_size = (num_rows * 256,)
    from core.queue_registry import DeviceQueueRegistry
    active_queue = DeviceQueueRegistry.get_current()
    # Запускаем ядро, явно проверяя, что передаем нативные cl.Buffer объекты
    self.prg_softmax.softmax_forward(
        active_queue,
        global_size,
        local_size,
        x.data_buffer,
        y.data_buffer,
        np.int32(len_)
    )
    active_queue.finish()
    return y

```

```python
def softmax_backward(self, grad_output: Tensor, y: Tensor) -> Tensor:
    """
    Обратный проход глобальной операции Softmax.
    Использует уже вычисленный forward-выход тензора Y для экономии памяти VRAM.

    :param grad_output: Пришедший сверху градиент dL/dY [B * H * L, SEQ_LEN]
    :param y: Выход прямого прохода Softmax (вероятности) [B * H * L, SEQ_LEN]
    :return: Выходной градиент dL/dX [B * H * L, SEQ_LEN]
    """
    seq_len = y.shape[-1]
    num_rows = y.size // seq_len

    # Выделяем тензор для результирующего градиента по входу
    grad_input = Tensor(y.shape, self.ctx, self.queue)

    # Настройка сетки потоков (идентично forward шагу)
    local_size = (256,)
    global_size = (num_rows * 256,)
    from core.queue_registry import DeviceQueueRegistry
    active_queue = DeviceQueueRegistry.get_current()
    self.prg_softmax.softmax_backward(
        active_queue, global_size, local_size,
        grad_output.data_buffer, y.data_buffer, grad_input.data_buffer,
        np.int32(seq_len)
    )
    active_queue.finish()

    return grad_input
```

RSNORM

```
// BLOCK_SIZE — количество потоков в рабочей группе (например, 256)
// D_MODEL — размерность скрытого состояния (например, 400 или 4096)

__kernel void rmsnorm_forward(
    __global const float* X,       // Входной тензор [B * L, D_MODEL]
    __global const float* gamma,   // Обучаемые веса [D_MODEL]
    __global float* Y,             // Выходной тензор [B * L, D_MODEL]
    __global float* inv_rms_cache, // Кэш для Backward: 1/RMS для каждой строки [B * L]
    const int d_model)
{ ···
}
__kernel void rmsnorm_backward(
    __global const float* grad_output, // Пришедший градиент dL/dY [B * L, D_MODEL]
    __global const float* X,           // Исходный вход X [B * L, D_MODEL]
    __global const float* gamma,       // Веса [D_MODEL]
    __global const float* inv_rms_cache,// Сохраненный инвертированный RMS [B * L]
    __global float* grad_input,        // Выходной градиент dL/dX [B * L, D_MODEL]
    __global float* grad_gamma_part,   // Промежуточный градиент по весам (для редукции)
    const int d_model)
{ ···
}
```

```
@auto_backward
def rmsnorm(self, x: Tensor, gamma: Tensor) -> Tensor:
    """
    Глобальная операция RMSNorm (Forward)
    Обрабатывает тензор любой формы, считая нормализацию по последней оси d_model
    """
    d_model = x.shape[-1]
    # Вычисляем количество строк (B * L)
    num_rows = x.size // d_model

    # Создаем выходной тензор и кэш инвертированного RMS для будущего backward
    y = Tensor(x.shape, self.ctx, self.queue)
    inv_rms_cache = cl.Buffer(self.ctx, cl.mem_flags.READ_WRITE, num_rows * 4) # float32 = 4 bytes

    # Настройка сетки потоков
    # Каждая рабочая группа из 256 потоков обрабатывает одну строку
    local_size = (256,)
    global_size = (num_rows * 256,)
    from core.queue_registry import DeviceQueueRegistry
    active_queue = DeviceQueueRegistry.get_current()
    self.prg_rmsnorm.rmsnorm_forward(
        active_queue, global_size, local_size,
        x.data_buffer, gamma.data_buffer, y.data_buffer, inv_rms_cache,
        np.int32(d_model)
    )
    active_queue.finish()

    # Сохраняем кэш редукции прямо внутри тензора (нужно для обучения)
    y._inv_rms_cache = inv_rms_cache
    return y
```

```python
    def rmsnorm_backward(self, grad_output: Tensor, x: Tensor, gamma: Tensor) -> tuple[Tensor, Tensor]:
        """
        Обратный проход глобальной операции RMSNorm.

        :param grad_output: Пришедший сверху градиент dL/dY
        :param x: Исходный входной тензор X из прямого прохода
        :param gamma: Обучаемые веса слоя gamma
        :return: Кортеж (grad_input, grad_gamma)
        """
        d_model = x.shape[-1]
        num_rows = x.size // d_model

        # 1. Выделяем тензоры для градиентов по входу (X) и весам (gamma)
        grad_input = Tensor(x.shape, self.ctx, self.queue)
        grad_gamma = Tensor(gamma.shape, self.ctx, self.queue)
        # Вспомогательный буфер для накопления градиентов по gamma от каждой строки
        grad_gamma_part = cl.Buffer(self.ctx, cl.mem_flags.READ_WRITE, x.size * 4) # float32

        # Извлекаем сохраненный на этапе forward кэш inv_rms
        # Если вызов идет из цепочки модели, кэш привязан к промежуточному выходу прямого прохода
        assert hasattr(grad_output, '_inv_rms_cache') or hasattr(x, '_inv_rms_cache'), \
            "Кэш inv_rms_cache не найден. Прямой проход rmsnorm должен быть выполнен до backward."
        inv_rms_cache = getattr(grad_output, '_inv_rms_cache', getattr(x, '_inv_rms_cache', None))

        # 2. Запуск Си-ядра вычисления градиента по входу
        local_size = (256,)
        global_size = (num_rows * 256,)
        from core.queue_registry import DeviceQueueRegistry
        active_queue = DeviceQueueRegistry.get_current()
        self.prg_rmsnorm.rmsnorm_backward(
            active_queue, global_size, local_size,
            grad_output.data_buffer, x.data_buffer, gamma.data_buffer, inv_rms_cache,
            grad_input.data_buffer, grad_gamma_part,
            np.int32(d_model)
        )
        active_queue.finish()
        # 3. Редукция градиента по обучаемым весам gamma
        # Так как веса gamma общие для всего батча и последовательности, нам нужно
        # просуммировать промежуточный буфер grad_gamma_part по оси строк.
        # Для простоты и точности сделаем редукцию на GPU или вызовем быстрое ядро.
        # В рамках мета-программы на этапе тестов мы можем временно саккумулировать это:
        # (В продакшене LLM здесь вызывается простое редукционное ядро суммы столбцов матрицы)
        host_parts = np.empty((num_rows, d_model), dtype=np.float32)
        cl.enqueue_copy(active_queue, host_parts, grad_gamma_part)
        active_queue.finish()
        # Суммируем градиенты от всех токенов в единый вектор градиента весов
        final_grad_gamma = np.sum(host_parts, axis=0, dtype=np.float32)

        # Загружаем финальный градиент обратно во VRAM буфер grad_gamma
        cl.enqueue_copy(active_queue, grad_gamma.data_buffer, final_grad_gamma)
        active_queue.finish()

        return grad_input, grad_gamma
```

BMM

C: > Users > Дмитрий > github_repo > my_pytchon_opencl_lib > cl_kernels > matmul.cl

```
#define TILE_SIZE 16

// Ядро классического матричного умножения двух двумерных матриц с использованием локальной памяти.
// Матрица A имеет размерность [M x K]
// Матрица B имеет размерность [K x N]
// Выходная матрица C имеет размерность [M x N]

__kernel void matmul_tiled(
    __global const float* A,
    __global const float* B,
    __global float* C,
    const int M,
    const int K,
    const int N)
{ ···
}
#define TILE_SIZE 16

// Ядро пакетного умножения матриц (Batch Matrix Multiplication)
// A имеет форму [Batch, M, K]
// B имеет форму [Batch, K, N]
// C имеет форму [Batch, M, N]
__kernel void bmm_tiled(
    __global const float* A,
    __global const float* B,
    __global float* C,
    const int M,
    const int K,
    const int N,
    const int batch_stride_A, // M * K
    const int batch_stride_B, // K * N
    const int batch_stride_C) // M * N
{ ···
}
```

```
    @auto_backward
    def bmm(self, A: Tensor, B: Tensor) -> Tensor:
        """
        Пакетное матричное умножение (Batch Matrix Multiplication)
        A: [Batch, M, K], B: [Batch, K, N] -> Выход: [Batch, M, N]
        """
        assert len(A.shape) == 3 and len(B.shape) == 3, "BMM ожидает строго 3D-тензоры [Batch, M/K, K/N]"
        assert A.shape[0] == B.shape[0], "Размерность Batch у матриц A и B должна совпадать"
        assert A.shape[2] == B.shape[1], f"Несовпадение внутренних осей матриц: K={A.shape[2]} и K={B.shape[1]}"

        Batch = A.shape[0]
        M, K = A.shape[1], A.shape[2]
        N = B.shape[3] if len(B.shape) > 3 else B.shape[2]

        C = Tensor([Batch, M, N], self.ctx, self.queue)

        # Шаги смещения внутри плоского буфера для каждой матрицы пакета
        batch_stride_A = M * K
        batch_stride_B = K * N
        batch_stride_C = M * N

        # Настройка 3D-сетки потоков (под тайлы 16x16)
        local_size = (16, 16, 1)
        global_size_x = int(np.ceil(N / 16.0) * 16)
        global_size_y = int(np.ceil(M / 16.0) * 16)
        global_size_z = Batch
        global_size = (global_size_x, global_size_y, global_size_z)
        from core.queue_registry import DeviceQueueRegistry
        active_queue = DeviceQueueRegistry.get_current()
        self.prg_matmul.bmm_tiled(
            active_queue, global_size, local_size,
            A.data_buffer, B.data_buffer, C.data_buffer,
            np.int32(M), np.int32(K), np.int32(N),
            np.int32(batch_stride_A), np.int32(batch_stride_B), np.int32(batch_stride_C)
        )
        active_queue.finish()

        return C
```

```python
def bmm_backward(self, grad_output: Tensor, A: Tensor, B: Tensor) -> tuple[Tensor, Tensor]:
    """
    Обратный проход глобальной операции пакетного матричного умножения (BMM).
    Реализует дифференцирование на основе переиспользования ядра bmm_tiled.

    :param grad_output: Пришедший сверху градиент dL/dC [Batch, M, N]
    :param A: Исходный входной тензор A прямого прохода [Batch, M, K]
    :param B: Исходный входной тензор B прямого прохода [Batch, K, N]
    :return: Кортеж (grad_A, grad_B) с размерностями [Batch, M, K] и [Batch, K, N]
    """
    # Извлекаем геометрию исходных тензоров
    Batch, M, K = A.shape
    _, _, N = B.shape

    # 1. Вычисляем grad_A = grad_output @ B.T
    # Для транспонирования последних двух осей матрицы B ([Batch, K, N] -> [Batch, N, K])
    # используем наше универсальное и быстрое ядро permute
    B_T = self.permute(B, [0, 2, 1])

    # Вызываем bmm_tiled: [Batch, M, N] @ [Batch, N, K] -> [Batch, M, K]
    grad_A = self.bmm(grad_output, B_T)

    # 2. Вычисляем grad_B = A.T @ grad_output
    # Транспонируем последние две оси матрицы A ([Batch, M, K] -> [Batch, K, M])
    A_T = self.permute(A, [0, 2, 1])

    # Вызываем bmm_tiled: [Batch, K, M] @ [Batch, M, N] -> [Batch, K, N]
    grad_B = self.bmm(A_T, grad_output)

    return grad_A, grad_B
```

PERMUTE

Softmax.cl | nncl_glob.py | matmul.cl | permute.cl | RMSNorm.cl

C: > Users > Дмитрий > github_repo > my_pytchon_opencl_lib > cl_kernels > permute.cl

```
// Распаковка линейного индекса в координаты по заданной форме
inline void get_coords_permute(int linear_idx, __global const int* dims, int num_dims, int* out_coords) {…
}

// Кернел универсальной перестановки осей тензора любой размерности (до 10D)
__kernel void tensor_permute(
    __global const float* src,
    __global float* dst,
    __global const int* src_dims,
    __global const int* dst_dims,
    __global const int* permute_axes, // Пример для x.transpose(1, 2): [0, 2, 1, 3]
    const int num_dims,
    const int total_elements)
{…
}
```

```python
@auto_backward
def permute(self, x: Tensor, axes: list) -> Tensor:
    """
    Глобальная перестановка осей тензора (Универсальный Permute/Transpose до 10D)
    """
    assert len(axes) == len(x.shape), "Количество осей перестановки должно совпадать с ndim тензора"

    # Вычисляем новую форму выходного тензора
    dst_shape = [x.shape[i] for i in axes]
    y = Tensor(dst_shape, self.ctx, self.queue)

    # Переводим метаданные в C-массивы int32
    src_dims_g = cl.Buffer(self.ctx, cl.mem_flags.READ_ONLY | cl.mem_flags.COPY_HOST_PTR, hostbuf=np.array(x.shape, dtype=np.int32))
    dst_dims_g = cl.Buffer(self.ctx, cl.mem_flags.READ_ONLY | cl.mem_flags.COPY_HOST_PTR, hostbuf=np.array(dst_shape, dtype=np.int32))
    axes_g = cl.Buffer(self.ctx, cl.mem_flags.READ_ONLY | cl.mem_flags.COPY_HOST_PTR, hostbuf=np.array(axes, dtype=np.int32))

    global_size = (x.size,)
    from core.queue_registry import DeviceQueueRegistry
    active_queue = DeviceQueueRegistry.get_current()
    self.kernel_permute(
        active_queue, global_size, None,
        x.data_buffer, y.data_buffer,
        src_dims_g, dst_dims_g, axes_g,
        np.int32(x.ndim), np.int32(x.size)
    )
    active_queue.finish()
    return y
```

CASUAL_CONV

C: > Users > Дмитрий > github_repo > my_pytchon_opencl_lib > cl_kernels > CausalConv1d.cl

```
// Входной тензор имеет форму [Batch, Channels, Length] (после вашего transpose)
// Веса имеют форму [Channels, Kernel_Size] (так как свертка depthwise)
// Bias имеет форму [Channels]

__kernel void causal_conv1d_depthwise(
    __global const float* X,      // [Batch, Channels, Length]
    __global const float* weight, // [Channels, Kernel_Size]
    __global const float* bias,   // [Channels]
    __global float* Y,            // [Batch, Channels, Length]
    const int batch_size,
    const int channels,
    const int length,
    const int kernel_size)
{ ···
}

// Ядро 1: Вычисление градиента по входу (grad_X).
// Выполняет зеркальный (антикаузальный) проход по времени.
__kernel void causal_conv1d_backward_input(
    __global const float* grad_output, // [Batch, Channels, Length]
    __global const float* weight,      // [Channels, Kernel_Size]
    __global float* grad_X,            // [Batch, Channels, Length]
    const int batch_size,
    const int channels,
    const int length,
    const int kernel_size)
{ ···
}

// Ядро 2: Вычисление градиента по весам (grad_weight) и bias.
// Каждый поток отвечает за конкретную ячейку фильтра конкретного канала.
// Потоки суммируют вклады по всей оси времени и по всему батчу.
__kernel void causal_conv1d_backward_weights(
    __global const float* grad_output, // [Batch, Channels, Length]
    __global const float* X,           // [Batch, Channels, Length]
    __global float* grad_weight,       // [Channels, Kernel_Size]
    __global float* grad_bias,         // [Channels]
    const int batch_size,
    const int channels,
    const int length,
    const int kernel_size)
{ ···
}
```

```
    @auto_backward
    def causal_conv1d(self, x: Tensor, weight: Tensor, bias: Tensor) -> Tensor:
        """
        Каузальная Depthwise свертка (Forward)
        x: [Batch, Channels, Length]
        weight: [Channels, Kernel_Size]
        bias: [Channels]
        """
        Batch, Channels, Length = x.shape
        kernel_size = weight.shape[1]

        y = Tensor(x.shape, self.ctx, self.queue)

        # 3D сетка потоков: по всей временной оси, по каналам, по батчу
        global_size = (Length, Channels, Batch)
        from core.queue_registry import DeviceQueueRegistry
        active_queue = DeviceQueueRegistry.get_current()
        self.prg_conv.causal_conv1d_depthwise(
            active_queue, global_size, None,
            x.data_buffer, weight.data_buffer, bias.data_buffer, y.data_buffer,
            np.int32(Batch), np.int32(Channels), np.int32(Length), np.int32(kernel_size)
        )
        active_queue.finish()
        return y
    ##################################################
```

```python
    def causal_conv1d_backward(self, grad_output: Tensor, x: Tensor, weight: Tensor) -> tuple[Tensor, Tensor, Tensor]:
        """
        Обратный проход глобальной каузальной Depthwise свертки.

        :param grad_output: Градиент dL/dY [Batch, Channels, Length]
        :param x: Исходный входной тензор X из прямого прохода [Batch, Channels, Length]
        :param weight: Исходные веса свертки [Channels, Kernel_Size]
        :return: Кортеж (grad_X, grad_weight, grad_bias)
        """
        Batch, Channels, Length = x.shape
        kernel_size = weight.shape[1] # Извлекаем Kernel_Size (вторая ось формы весов)

        # Выделяем выходные тензоры под градиенты
        grad_X = Tensor(x.shape, self.ctx, self.queue)
        grad_weight = Tensor(weight.shape, self.ctx, self.queue)
        grad_bias = Tensor([Channels], self.ctx, self.queue) # ИСПРАВЛЕНО: добавлено self.ctx

        # 1. Запуск ядра расчета градиента по входу (3D Grid)
        # ИСПРАВЛЕНО: Обращаемся к объединенной программе self.prg_conv
        global_size_in = (Length, Channels, Batch)
        from core.queue_registry import DeviceQueueRegistry
        active_queue = DeviceQueueRegistry.get_current()
        self.prg_conv.causal_conv1d_backward_input(
            active_queue, global_size_in, None,
            grad_output.data_buffer, weight.data_buffer, grad_X.data_buffer,
            np.int32(Batch), np.int32(Channels), np.int32(Length), np.int32(kernel_size)
        )

        # 2. Запуск ядра расчета градиента по параметрам свертки (2D Grid)
        # ИСПРАВЛЕНО: Обращаемся к объединенной программе self.prg_conv
        global_size_w = (kernel_size, Channels)
        self.prg_conv.causal_conv1d_backward_weights(
            active_queue, global_size_w, None,
            grad_output.data_buffer, x.data_buffer, grad_weight.data_buffer, grad_bias.data_buffer,
            np.int32(Batch), np.int32(Channels), np.int32(Length), np.int32(kernel_size)
        )

        active_queue.finish()
        return grad_X, grad_weight, grad_bias
```

Embeding

```
// Кастомная атомарная функция для float, совместимая со спецификацией OpenCL 1.2
inline void atomic_add_float(__global volatile float* address, const float val)
{...
}

// Forward шаг: Выборка эмбеддингов по ID токенов
__kernel void embedding_forward(
    __global const int* X_tokens,
    __global const float* weight,
    __global float* Y_out,
    const int embedding_dim,
    const int total_tokens)
{...
}

// Backward шаг: Безопасная аккумуляция градиентов по матрице весов
__kernel void embedding_backward(
    __global const float* grad_output,
    __global const int* X_tokens,
    __global float* grad_weight,
    const int embedding_dim,
    const int total_tokens)
{...
}
```

```
    def embedding(self, x_tokens_buf, weight: Tensor, total_tokens: int) -> Tensor:
        """Прямой ход Embedding: Принимает total_tokens в явном виде"""
        vocab_size, embedding_dim = weight.shape

        # Создаем выходной тензор строго под реальный размер осей (8 * 16 = 128 элементов)
        y = Tensor([total_tokens, embedding_dim], self.ctx, self.queue)

        # Точная 2D-сетка потоков
        global_size = (total_tokens, embedding_dim)
        from core.queue_registry import DeviceQueueRegistry
        active_queue = DeviceQueueRegistry.get_current()
        self.prg_embedding.embedding_forward(
            active_queue, global_size, None,
            x_tokens_buf, weight.data_buffer, y.data_buffer,
            np.int32(embedding_dim), np.int32(total_tokens)
        )
        active_queue.finish()
        return y

    def embedding_backward(self, grad_output: Tensor, x_tokens_buf, weight_shape, total_tokens: int) -> Tensor:
        """Обратный ход Embedding"""
        vocab_size, embedding_dim = weight_shape

        grad_weight = Tensor(weight_shape, self.ctx, self.queue)
        cl.enqueue_fill_buffer(self.queue, grad_weight.data_buffer, np.float32(0.0), 0, grad_weight.nbytes)

        global_size = (total_tokens, embedding_dim)
        from core.queue_registry import DeviceQueueRegistry
        active_queue = DeviceQueueRegistry.get_current()
        self.prg_embedding.embedding_backward(
            active_queue, global_size, None,
            grad_output.data_buffer, x_tokens_buf, grad_weight.data_buffer,
            np.int32(embedding_dim), np.int32(total_tokens)
        )
        active_queue.finish()
        return grad_weight
```